



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2
по курсу «Разработка параллельных и
распределенных программ»

*Параллельная реализация решения
системы линейных алгебраических уравнений с помощью MPI*

Студент: Федуков А. А.

Группа: ИУ9-52Б

Преподаватель: Царев А. С.

Москва, 2025 г.

Содержание

Постановка задачи	2
РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВ- НЕНИЙ ИТЕРАЦИОННЫМИ МЕТОДАМИ	2
Практическая реализация	2
Код программы	3
Вывод программы	7
Вывод	10
Характеристики компьютера	10

Постановка задачи

РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ ИТЕРАЦИОННЫМИ МЕТОДАМИ

Пусть дана система из N линейных алгебраических уравнений в виде $Ax = b$, где A — матрица коэффициентов размером $N \times N$, b — вектор правых частей размером N , x — искомый вектор решений размером N .

Решение системы уравнений итерационным методом состоит в выполнении следующих шагов:

1. Задаётся $x^{(0)}$ — произвольное начальное приближение решения (вектор с произвольными начальными значениями).
2. Приближение многократно улучшается с использованием формулы вида $x^{(n+1)} = f(x^{(n)})$, где функция f определяется используемым методом¹.
3. Процесс продолжается, пока не выполнится условие $g(x^{(n)}) < \varepsilon$, где функция g определяется используемым методом, а величина ε задаёт требуемую точность.

¹Например, методом Якоби, методом Зейделя и др.

Практическая реализация

Код программы

Листинг 1: Исходный код программы

```
1 use mpi::Count;
2 use mpi::collective::SystemOperation;
3 use mpi::datatype::PartitionMut;
4 use mpi::traits::*;
5 use rand::prelude::*;
6 use std::time::Instant;
7
8 // забирает оперативки N*N*8 bytes
9 const N: usize = 16384;
10
11 // критерий остановки
12 const EPS: f64 = 1e-7;
13
14 // шаг в итерации
15 const TAU: f64 = 0.1 / N as f64;
16 // const TAU: f64 = -0.1 / N as f64;
17
18 // максимальное число итераций
19 const MAX_ITERS: usize = 2_000_000;
20
21 fn main() {
22     // Инициализация MPI
23     let universe = mpi::initialize().unwrap();
24     let world = universe.world(); // коммуникатор всех процессов
25
26     let rank = world.rank() as usize; // номер текущего процесса
27     let p = world.size() as usize; // общее число процессов
28
29     // Вычисляем, сколько строк у каждого процесса
30     let base = N / p; // число строк на процесс
31     let rem = N % p; // число процессов, получающих на 1 строку больше
32     // 0..(rem-1) процессы получают по (base+1) строк
33     // rem..(p) процессы получают по base строк
34
35     // число строк, обрабатываемых текущим процессом
36     let local_n = base + if rank < rem { 1 } else { 0 };
37     // начальный номер строки для текущего процесса
38     let start_row = rank * base + rank.min(rem);
39 }
```

```

40 // counts/displs нужны для сборки b_full через
all_gather_varcount_into
41 let mut counts: Vec<Count> = vec![0; p]; // число элементов от каждо
го процесса
42 let mut displs: Vec<Count> = vec![0; p]; // смещения для каждого про
цесса
43 {
44     let mut off: Count = 0;
45     for r in 0..p {
46         let cnt = (base + if r < rem { 1 } else { 0 }) as Count;
47         counts[r] = cnt;
48         displs[r] = off;
49         off += cnt;
50     }
51 }
52
53 // вектор из всех 1.0. он должен быть решением системы
54 let x_true = vec![1.0_f64; N];
55
56 // A_local - локальный блок строк матрицы A: local_n строк по N стол
бцов
57 let mut a_local = vec![0.0_f64; local_n * N];
58
59 // Заполняем A
60 // Модельная задача с заданным решением: диагональные элементы = 2,
остальные = 1
61 for i_local in 0..local_n {
62     let i_global = start_row + i_local;
63     let row_offset = i_local * N;
64     for j in 0..N {
65         a_local[row_offset + j] = if i_global == j { 2.0 } else {
1.0 };
66     }
67 }
68
69 // b_local = A_local * x_true
70 // считаем локальный кусок вектора правой части
71 let mut b_local = vec![0.0_f64; local_n];
72 for i_local in 0..local_n {
73     let row_offset = i_local * N;
74     let mut sum = 0.0_f64;
75     for j in 0..N {
76         sum += a_local[row_offset + j] * x_true[j];
77     }
78     b_local[i_local] = sum;
79 }

```

```

80
81 // посчитаем полный вектор b
82 // b дублируется во всех процессах: соберём b_full через Allgatherv
83 let mut b_full = vec![0.0_f64; N];
84 {
85     // разбиваем b_full на части для каждого процесса
86     let mut part_b = PartitionMut::new(&mut b_full[..], &counts[..],
87     &displs[..]);
88     // собираем полное b_full из локальных кусков b_local
89     world.all_gather_varcount_into(&b_local[..], &mut part_b);
90     // MPI_Allgatherv
91 }
92
93 // посчитаем ||b||_2
94 let local_b2: f64 = b_local.iter().map(|v| v * v).sum();
95 let mut global_b2 = 0.0_f64;
96 world.all_reduce_into(&local_b2, &mut global_b2, SystemOperation::
97 sum());
98 let norm_b = global_b2.sqrt();
99
100 // ----- Реализация итеративного метода
101
102 // x хранится целиком в каждом процессе
103 let mut x_n_full = vec![0.0_f64; N];
104 let mut x_np1_full = vec![0.0_f64; N];
105 let mut x_np1_local = vec![0.0_f64; local_n];
106
107 // инициализируем rng
108 let mut rng = StdRng::seed_from_u64(123456789); // фиксированный
109 seed
110
111 // инициализируем x0 как случайные числа в диапазоне [-1, 1]
112 for i in 0..N {
113     x_n_full[i] = rng.gen_range(-1.0_f64..1.0_f64);
114 }
115
116 // Замер времени вычислений
117 world.barrier();
118 let t0 = Instant::now();
119
120 let mut iters_done: usize = 0;
121 let mut last_ratio: f64 = f64::INFINITY;
122
123 // итерируемся, пока не привысим MAX_ITERS или не достигнем EPS
124 for n_iter in 0..MAX_ITERS {
125     let mut local_r2 = 0.0_f64;

```

```

123
124 // вычисляем локальный кусок  $r$  и  $x_{\{n+1\}}$ 
125 for i_local in 0..local_n {
126     //  $Ax_n$  для строки i_local:
127     let row_offset = i_local * N;
128     let mut ax = 0.0_f64;
129     for j in 0..N {
130         ax += a_local[row_offset + j] * x_n_full[j];
131     }
132
133     //  $r_i = (Ax_n)_i - b_i$ 
134     let i_global = start_row + i_local;
135     let r_i = ax - b_full[i_global];
136
137     local_r2 += r_i * r_i;
138
139     // считаем  $x_{\{n+1\}}$  для этой строки
140     let x_i_n = x_n_full[i_global];
141     x_np1_local[i_local] = x_i_n - TAU * r_i;
142 }
143
144 let mut global_r2 = 0.0_f64;
145
146 // собираем  $\|r\|_2^2$  по всем процессам как сумму локальных вкла
147 дов
148 world.all_reduce_into(&local_r2, &mut global_r2, SystemOperation
149 ::sum());
150 // MPI_Allreduce + MPI_SUM
151
152 // вычисляем  $g(x_n) = \|r\|_2 / \|b\|_2$ 
153 last_ratio = global_r2.sqrt() / norm_b;
154
155 // собираем полный  $x_{\{n+1\}}$  во всех процессах из x_np1_local
156 {
157     let mut part_xnp1 = PartitionMut::new(&mut x_np1_full[..], &
158 counts[..], &displs[..]);
159     world.all_gather_varcount_into(&x_np1_local[..], &mut
160 part_xnp1);
161 // MPI_Allgatherv
162 }
163
164 // меняем указатели для следующей итерации
165 std::mem::swap(&mut x_n_full, &mut x_np1_full);
166
167 // проверяем условие остановки

```

```

165         if last_ratio < EPS {
166             break;
167         }
168     }
169
170     // ждем завершения всех процессов
171     world.barrier();
172     // замеряем время
173     let elapsed = t0.elapsed().as_secs_f64();
174
175     // Вывод результатов
176     if rank == 0 {
177         println!("N={}, p={}, EPS={:.3e}, TAU={:.6e}", N, p, EPS, TAU);
178         println!("iters={}, g(x_n)={:.6e}", iters_done, last_ratio);
179         println!("time_sec={:.6}", elapsed);
180         println!("Result x_n_full[0..3]: {:?}", &x_n_full[0..3]);
181     }
182
183     // выводим максимальную погрешность
184     if rank == 0 {
185         let mut max_err = 0.0_f64;
186         for i in 0..N {
187             let e = (x_n_full[i] - x_true[i]).abs();
188             if e > max_err {
189                 max_err = e;
190             }
191         }
192         println!("max_abs_error={:.6e}", max_err);
193     }
194     println!("Rank {} done.", rank);
195 }

```

Вывод программы

Инициализируем начальное значение вектора $x^{(0)}$ случайными числами в диапазоне $[-1, 1]$ с фиксированным seed для генератора случайных чисел, чтобы обеспечить воспроизводимость результатов.

- Размер системы: $N = 16384$.
- Критерий остановки: $\varepsilon = 1 \times 10^{-7}$.
- Шаг итерации: $\tau = \frac{0.1}{N}$.

- Максимальное число итераций: 2000000.
- Используемые числа процессов: $p = 1, 2, 4, 8, 16$.
- `iters` — количество выполненных итераций до достижения критерия остановки.
- `g(x_n)` — значение критерия остановки на последней итерации.
- `time_sec` — время выполнения программы в секундах.
- `max_abs_error` — максимальная абсолютная ошибка между вычисленным решением и истинным решением (вектор из всех 1.0).

Не удавалось запустить с 8 и 16 процессами без дополнительных флагов из-за недостатка слотов на системе. С использованием флага ‘`–use-hwthread-cpus`’ для 8 процессов и ‘`–oversubscribe`’ для 16 процессов программа успешно выполнялась.

Листинг 2: Вывод программы

```

1 const N: usize = 16384;
2 const EPS: f64 = 1e-7;
3 const TAU: f64 = 0.1 / N as f64;
4
5 > cargo build --release
6 > mpirun -np 1 target/release/lab2
7 N=16384, p=1, EPS=1.000e-7, TAU=6.103516e-6
8 iters=154, g(x_n)=9.969014e-8
9 time_sec=30.898948
10 Result x_n_full[0..3]: [0.9999999102791989, 0.9999999102791989,
    0.9999999102791989]
11 max_abs_error=8.972080e-8
12 Rank 0 done.
13 > mpirun -np 2 target/release/lab2
14 N=16384, p=2, EPS=1.000e-7, TAU=6.103516e-6
15 iters=154, g(x_n)=9.969014e-8
16 time_sec=16.331055
17 Result x_n_full[0..3]: [0.9999999102791989, 0.9999999102791989,
    0.9999999102791989]
18 max_abs_error=8.972080e-8
19 Rank 0 done.
20 Rank 1 done.
21 > mpirun -np 4 target/release/lab2

```



```

22 N=16384, p=4, EPS=1.000e-7, TAU=6.103516e-6
23 iters=154, g(x_n)=9.969014e-8
24 time_sec=10.134222
25 Result x_n_full[0..3]: [0.9999999102791989, 0.9999999102791989,
    0.9999999102791989]
26 max_abs_error=8.972080e-8
27 Rank 0 done.
28 Rank 1 done.
29 Rank 2 done.
30 Rank 3 done.
31 > mpirun -np 8 target/release/lab2
32 -----
33 There are not enough slots available in the system to satisfy the 8
34 slots that were requested by the application:
35 ...
36 > mpirun --use-hwthread-cpus -np 8 target/release/lab2
37
38 N=16384, p=8, EPS=1.000e-7, TAU=6.103516e-6
39 iters=154, g(x_n)=9.969014e-8
40 time_sec=8.730051
41 Result x_n_full[0..3]: [0.9999999102791989, 0.9999999102791989,
    0.9999999102791989]
42 max_abs_error=8.972080e-8
43 Rank 0 done.
44 Rank 1 done.
45 Rank 2 done.
46 Rank 3 done.
47 Rank 4 done.
48 Rank 5 done.
49 Rank 6 done.
50 Rank 7 done.
51 > mpirun --use-hwthread-cpus -np 16 target/release/lab2
52 There are not enough slots available in the system to satisfy the 16
53 slots that were requested by the application:
54 ...
55
56 > mpirun --oversubscribe -np 16 target/release/lab2
57
58 Rank 10 done.
59 Rank 11 done.
60 Rank 12 done.
61 Rank 14 done.
62 Rank 15 done.
63 Rank 1 done.
64 Rank 2 done.

```

```

65 Rank 4 done.
66 Rank 6 done.
67 Rank 8 done.
68 Rank 9 done.
69 Rank 3 done.
70 Rank 7 done.
71 Rank 13 done.
72 Rank 5 done.
73 N=16384, p=16, EPS=1.000e-7, TAU=6.103516e-6
74 iters=154, g(x_n)=9.969014e-8
75 time_sec=10.606535
76 Result x_n_full[0..3]: [0.9999999102791989, 0.9999999102791989,
    0.9999999102791989]
77 max_abs_error=8.972080e-8
78 Rank 0 done.

```

Вывод

В данной лабораторной работе была реализована параллельная программа на языке Rust с использованием библиотеки MPI для решения системы линейных алгебраических уравнений итерационным методом. Программа успешно выполняет распределённые вычисления, демонстрируя снижение времени выполнения при увеличении числа процессов. Результаты показывают, что достигнута требуемая точность решения, а максимальная абсолютная ошибка остаётся в пределах допустимых значений. Работа иллюстрирует эффективность параллельных вычислений для задач линейной алгебры и подчёркивает важность правильного распределения данных между процессами для оптимизации производительности.

Характеристики компьютера

Листинг 3: Характеристики компьютера

```

1 > inxi --cpu --memory
2 Memory:
3   System RAM: total: 16 GiB available: 14.8 GiB used: 10.48 GiB (70.8%)
4   Array-1: capacity: 64 GiB slots: 4 modules: 4 EC: None
5   Device-1: Channel-A DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
6   Device-2: Channel-B DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s

```

```

7   Device -3: Channel-C DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
8   Device -4: Channel-D DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
9 CPU:
10  Info: 6-core model: AMD Ryzen 5 7640HS w/ Radeon 760M Graphics bits:
    64
11    type: MT MCP cache: L2: 6 MiB
12  Speed (MHz): avg: 1200 min/max: 400/5017 cores: 1: 1390 2: 1100 3:
    1100
13    4: 1100 5: 1100 6: 1100 7: 1396 8: 1100 9: 1432 10: 1100 11: 1389
    12: 1100

```